



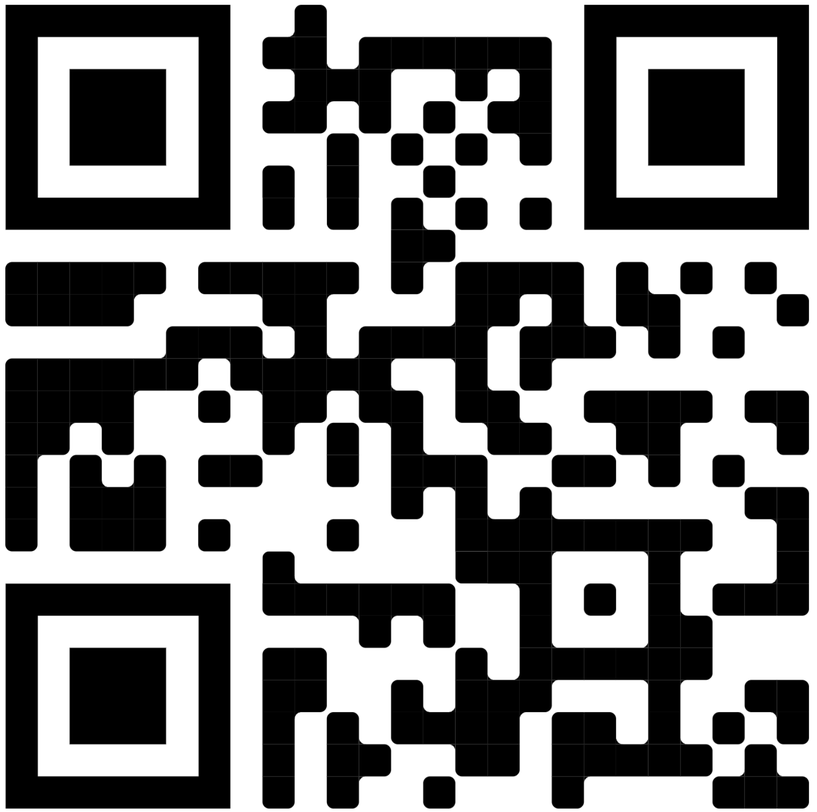
Distributed systems

Lectures: doc. dr. sc. Nikola Tanković

Replication



https://bit.ly/rs-blic-2



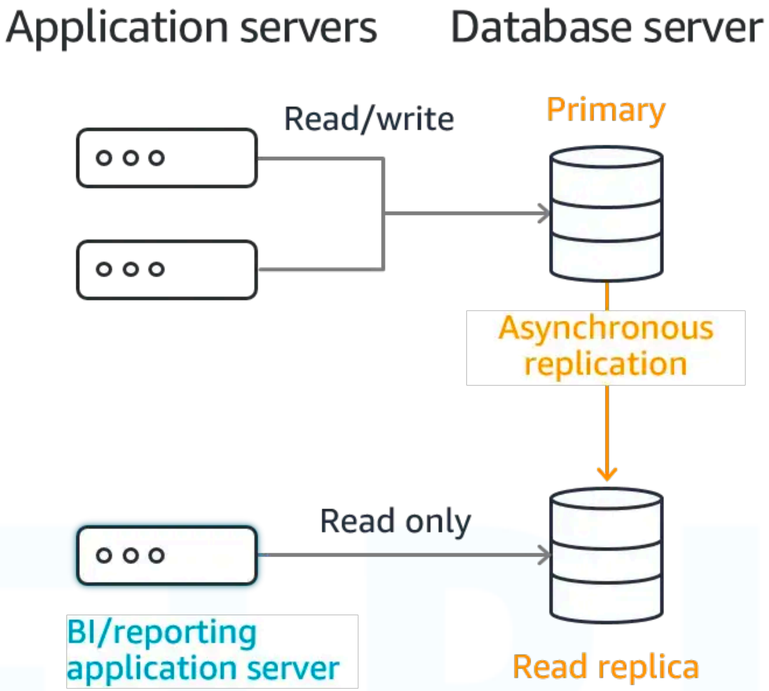
http://bit.ly/rs-tu-sam

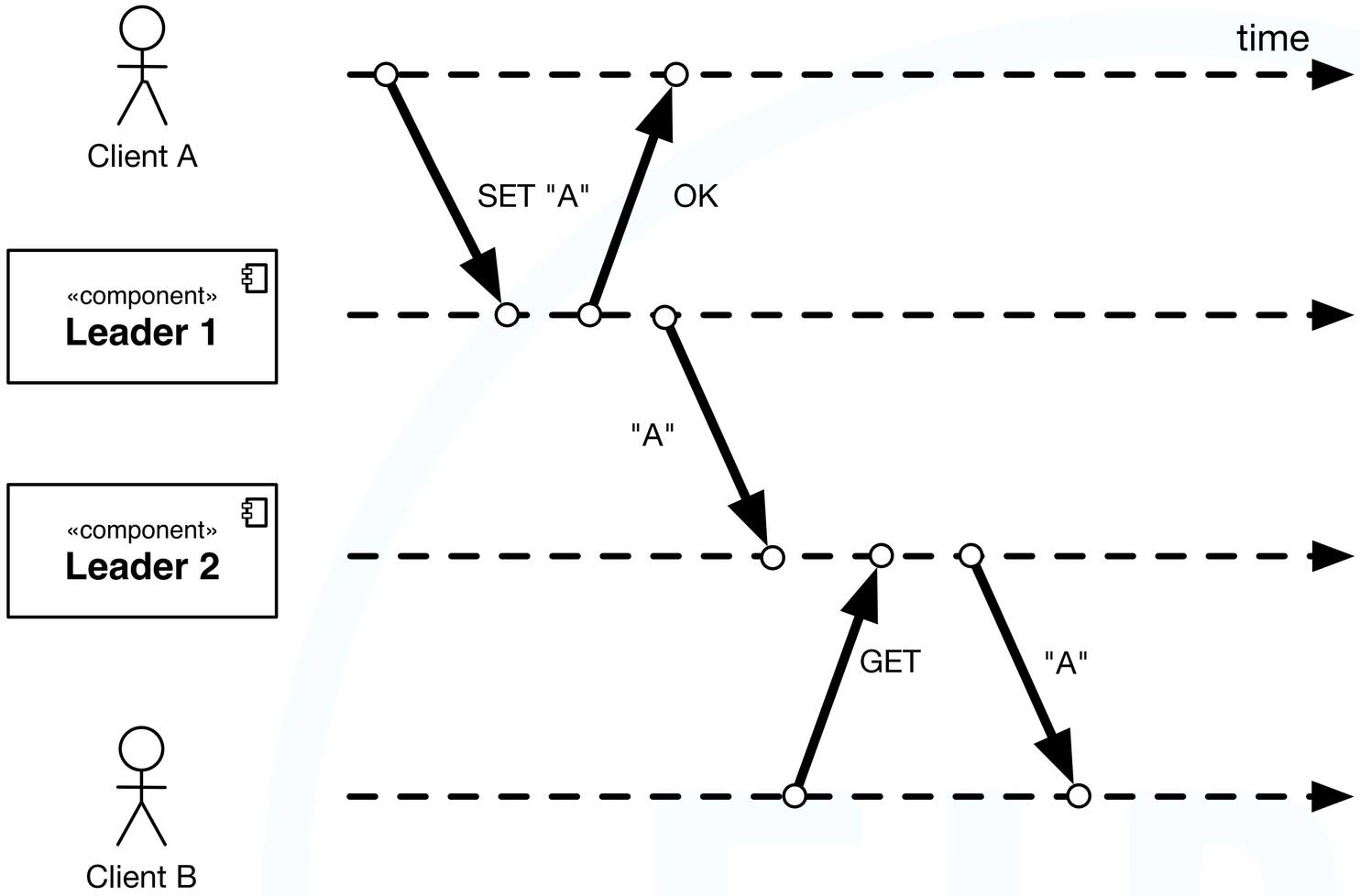
Replication

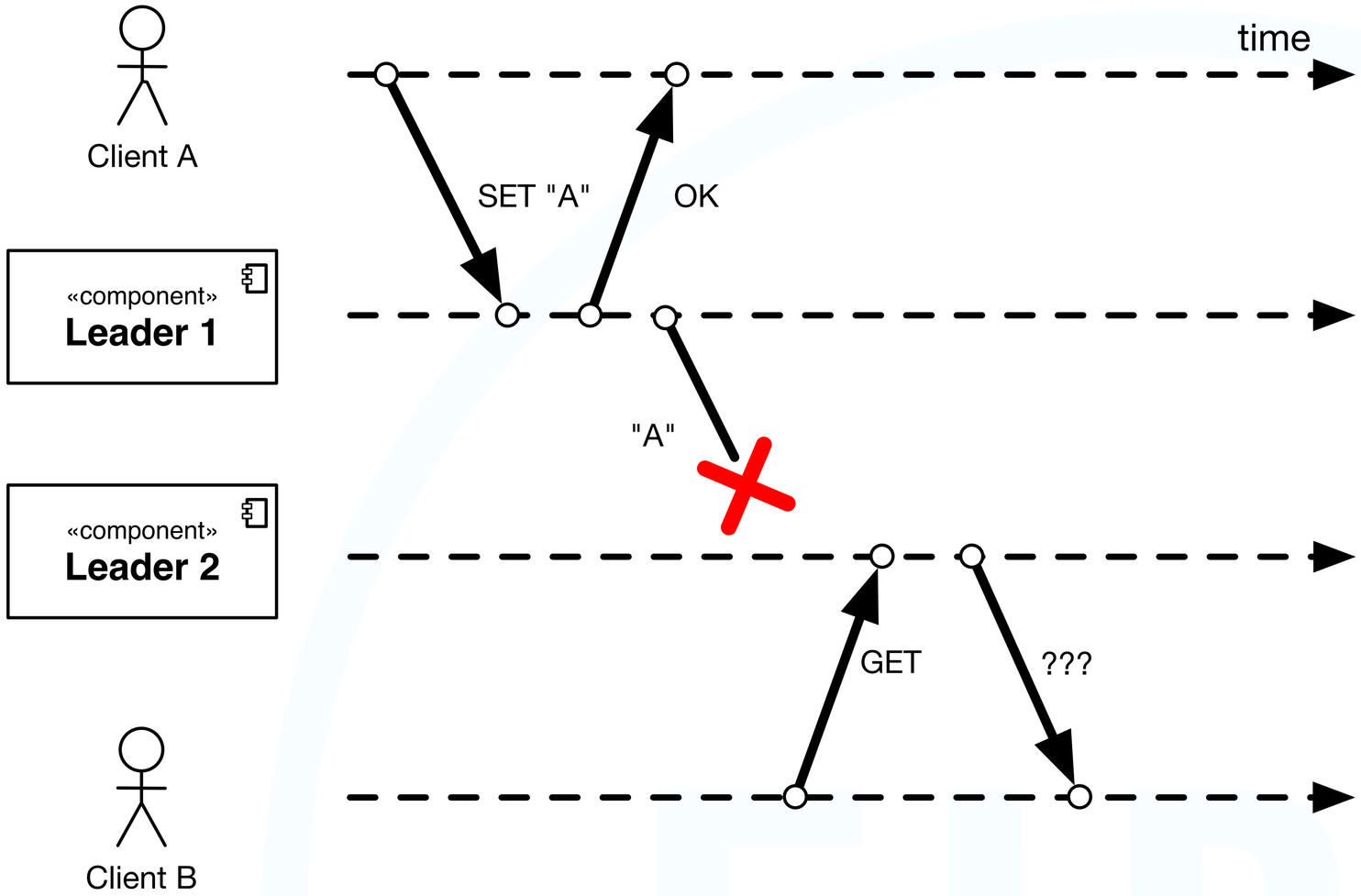
- Copy of the **same data** on **multiple nodes**
  - Databases
    - Apache **Cassandra** (Netflix, eBay, Uber, ...)
    - MongoDB (primary/replica)
    - Some of the SQL databases (*)
  - Filesystems
    - HDFS, NFS (Sun, 1984), ...
  - Caches
    - Redis, ...

Replica

- A copy of the data is called a **replica**
- If one replica is not available, others could still be available
- Challenges?

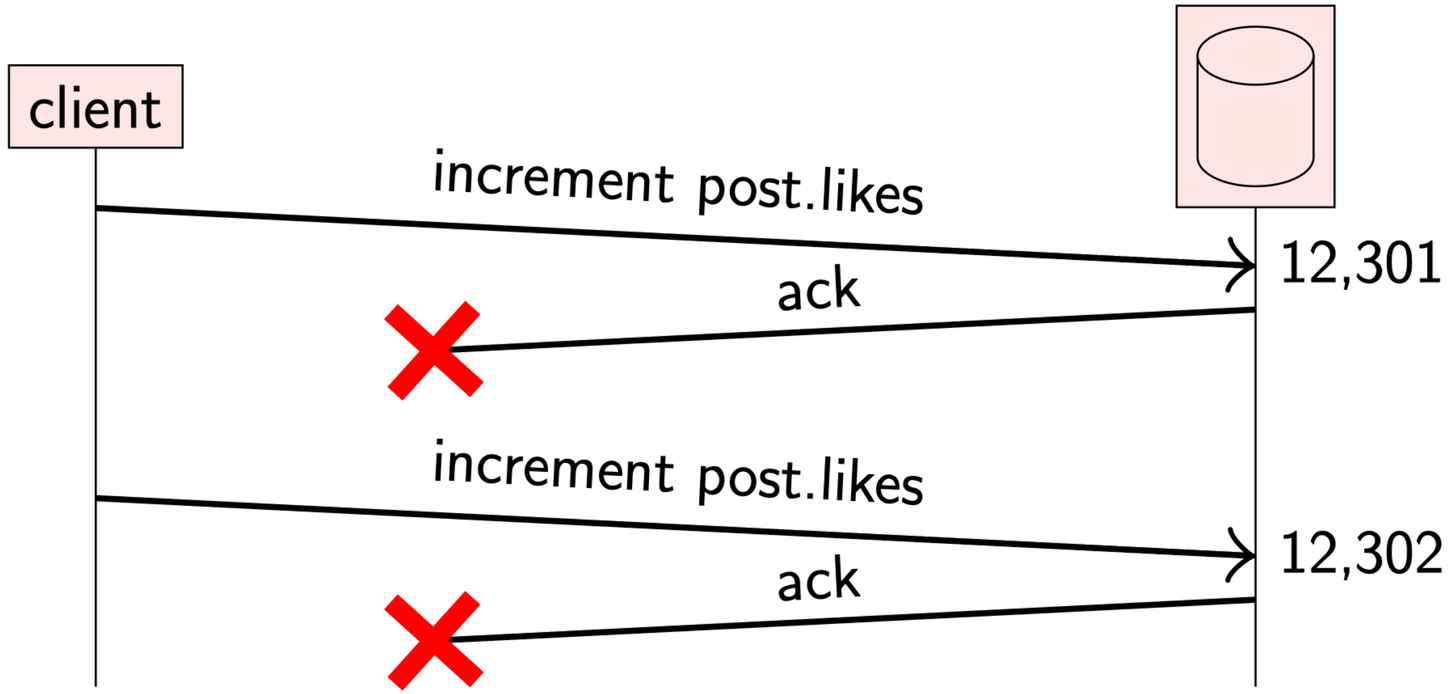



Client A
time
SET "A"
OK
«component»
Leader 1
"A"
«component»
Leader 2
GET
"A"
Client B


Client A
time
SET "A"
OK
«component»
Leader 1
"A"
«component»
Leader 2
GET
???
Client B

Synchronization

- … usual problems



Author: Dr. Martin Kleppmann

Retry behavior semantics

- **At-most-once:**
  - Send a message, no retries, hope for the best...
- **At-least-once:**
  - Retry until acknowledged, update can happen many times
- **Exactly-once:**
  - Retry but be careful not to have any side effects (!)
  - **Deduplicate** or **Idempotency**

Idempotency

- A function f is idempotent if

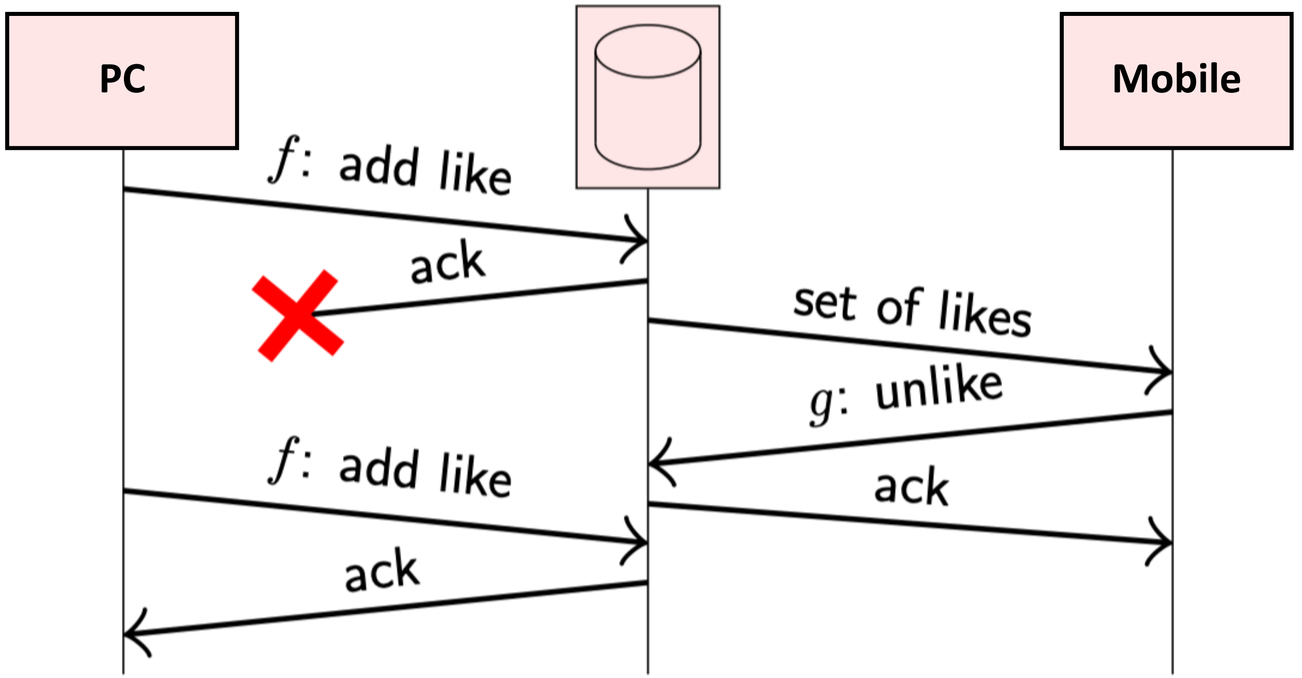
$$f(x) = f(f(x))$$

  - **Not ok:**

$$f_1(\textbf{like}) = \textbf{like} + 1$$

  - **Idempotent version:** 🤔

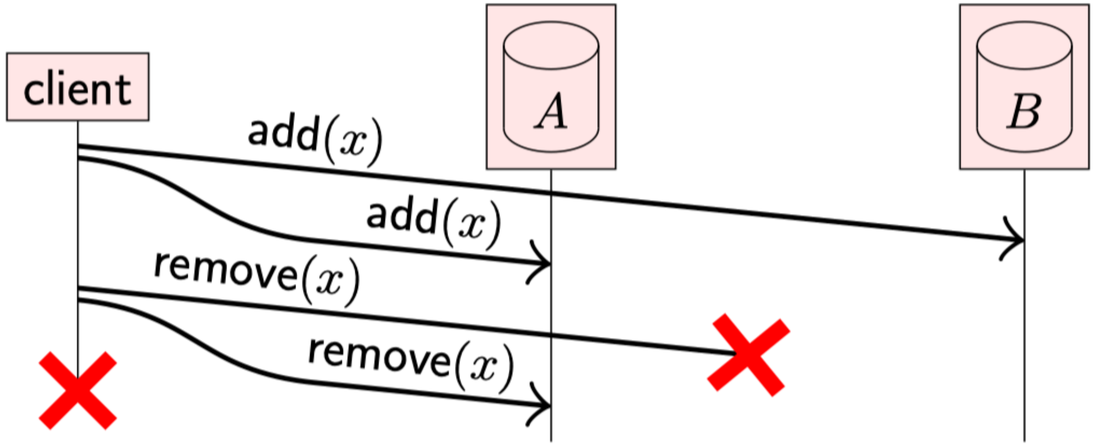
What about now?



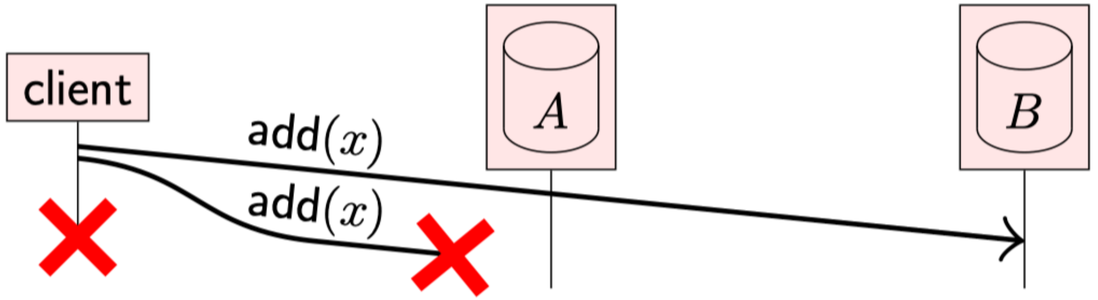
$f(likes) = likes \cup \{userID\}$
$g(likes) = likes \setminus \{userID\}$
Idempotent? $f(f(x)) = f(x)$ but $f(g(f(x)) \neq g(f(x))$

Author: Dr. Martin Kleppmann

Similar problem

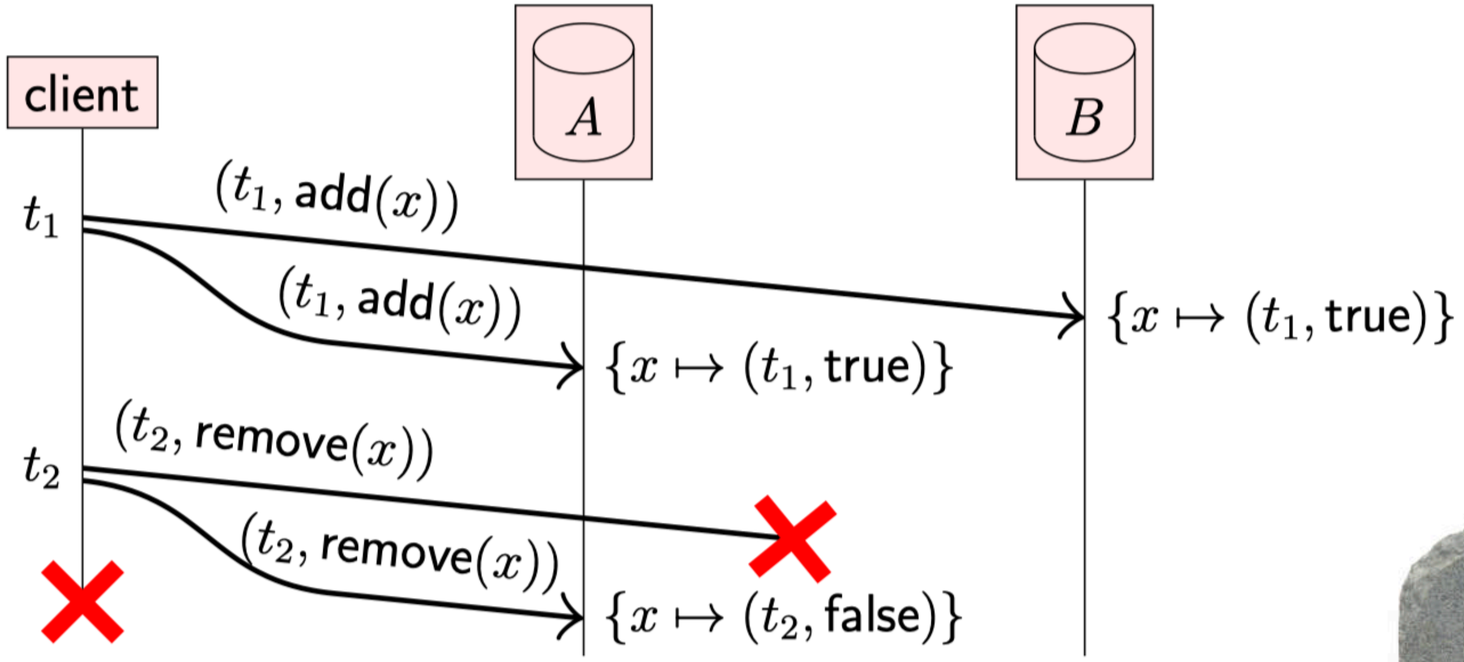


Final state ($x \notin A,\ x \in B$) is the same as in this case:



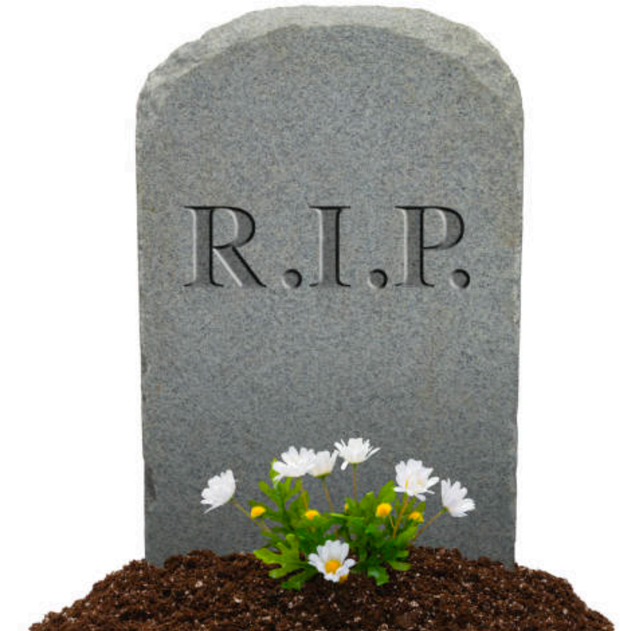
Author: Dr. Martin Kleppmann

Logical clock + Tombstone



"remove(x)" doesn't actually remove x: it labels x with "false" to indicate it is invisible (a **tombstone**)

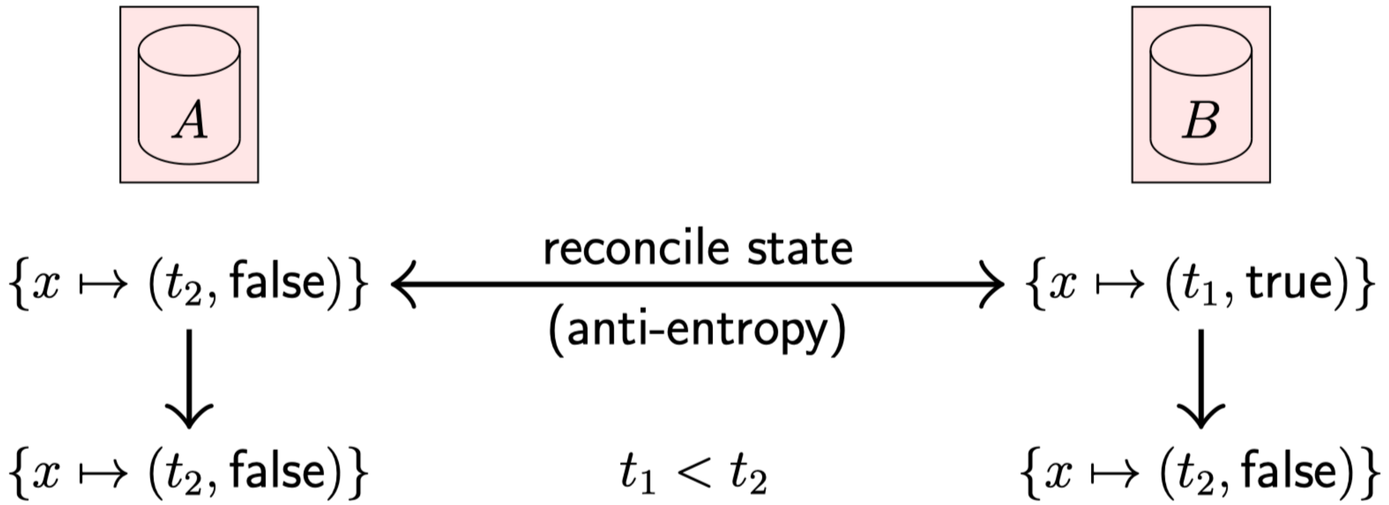
Every record has **logical timestamp** of last write



Author: Dr. Martin Kleppmann

Synchronizing replicas

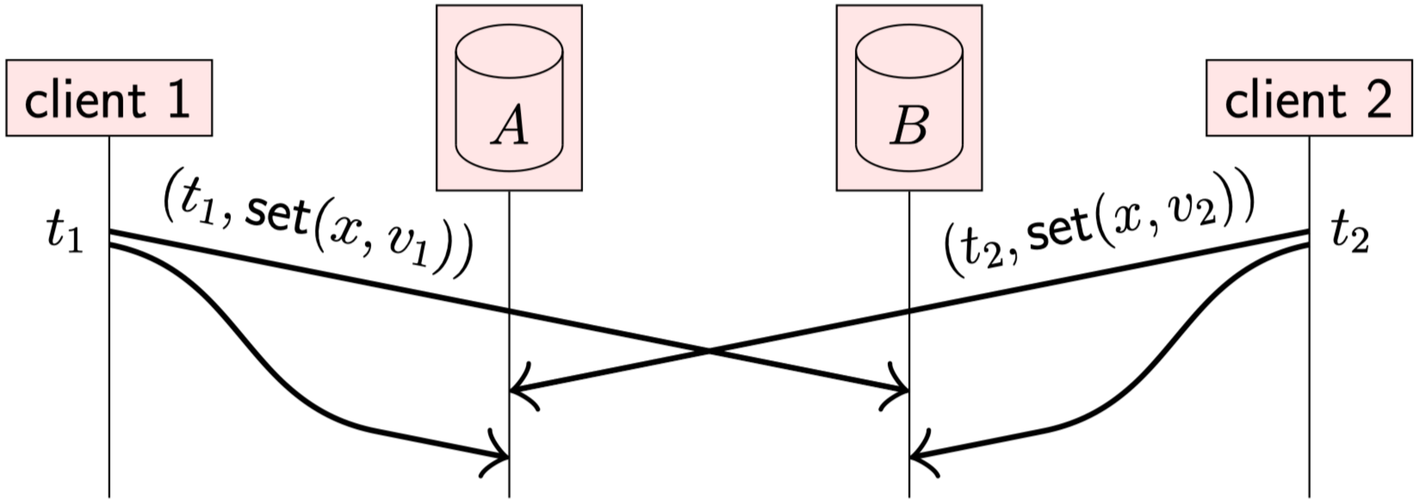
Replicas periodically communicate among themselves
to check for any inconsistencies.



Propagate the record with the latest timestamp,
discard the records with earlier timestamps
(for a given key).

Author: Dr. Martin Kleppmann

Concurrent writes

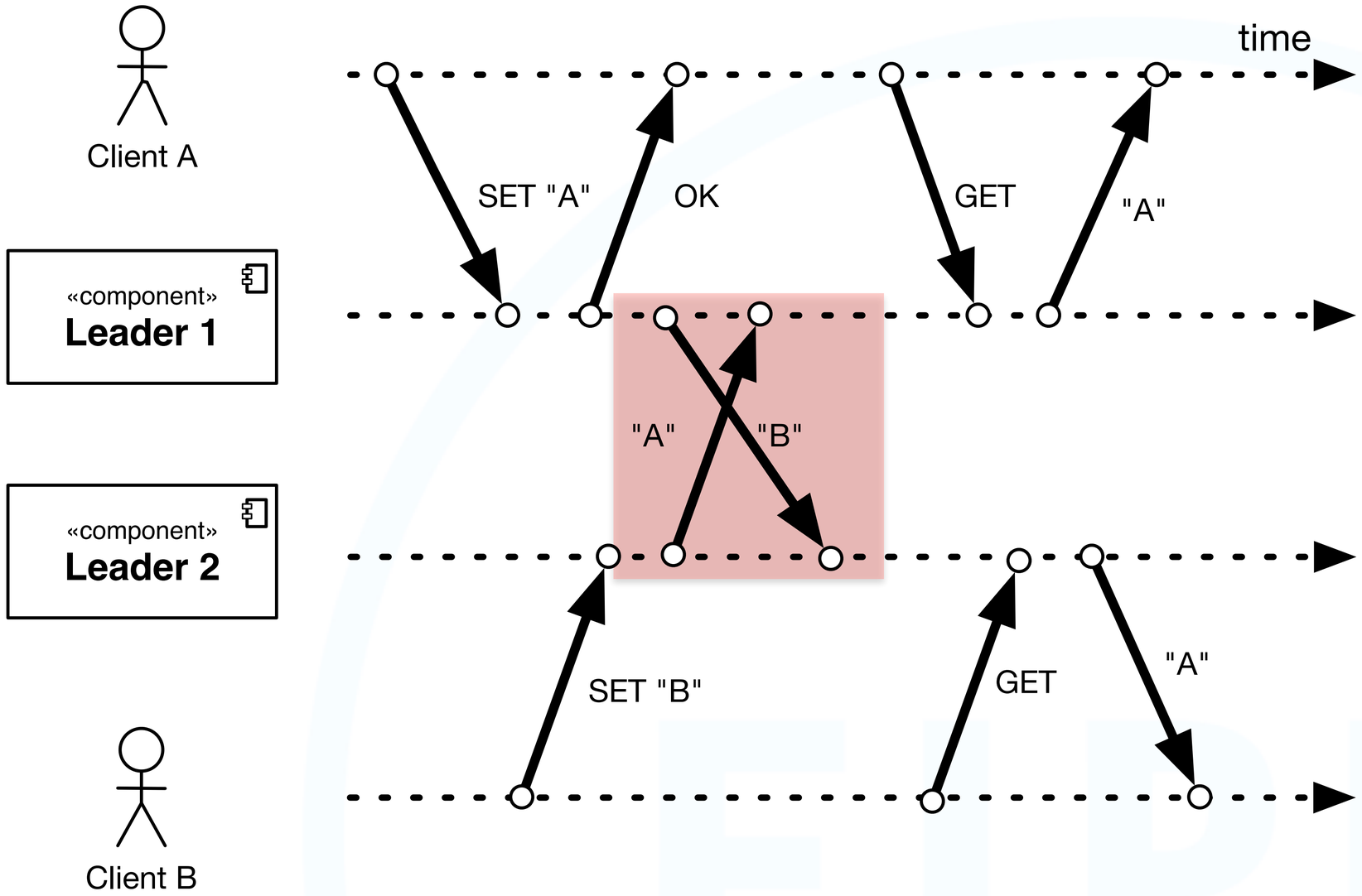


Two common approaches:

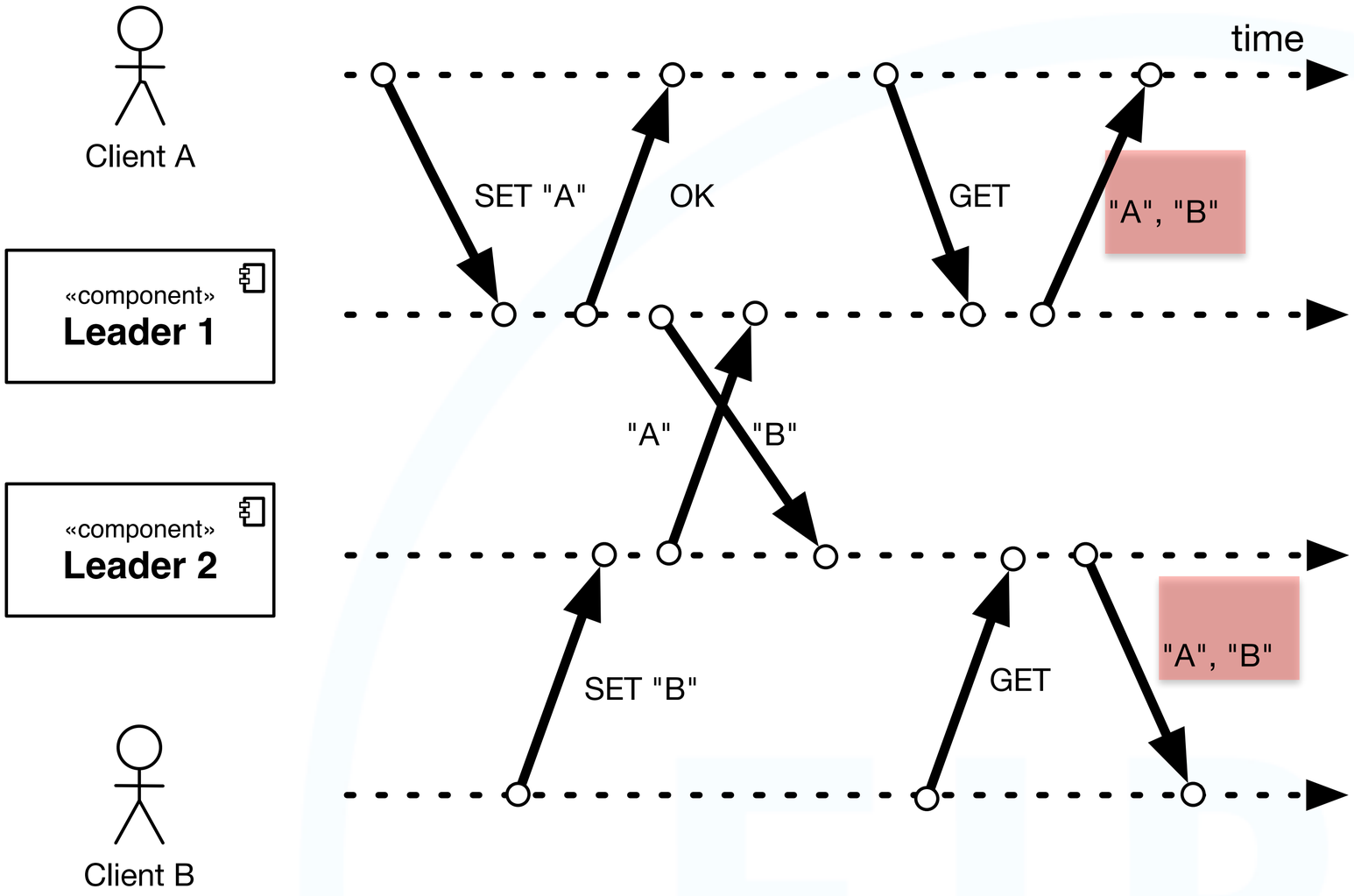
- **Last writer wins** (LWW):
  Use timestamps with total order (e.g. Lamport clock)
  Keep v_2 and discard v_1 if $t_2 > t_1$. Note: **data loss**!
- **Multi-value register**:
  Use timestamps with partial order (e.g. vector clock)
  v_2 replaces v_1 if $t_2 > t_1$; preserve both $\{v_1, v_2\}$ if $t_1 \parallel t_2$

Author: Dr. Martin Kleppmann

Last Writer Wins



Multi-Value Register



Challenges

- Vector clocks are expensive for a large number of clients
  - Can be larger than the data itself
- Alternatives exist
  - Dotted version vectors (2010) - used in Riak DB
- Apache Cassandra uses Physical clocks! 😳
- AWS Dynamo uses vector clocks

Fault probabilities

- A **replica** could be **unavailable** (network, hardware, ...)
- Assume p as the probability of the node being unavailable (p is not constant in time, but for example) as n replicas
- $P(\textbf{all down}) = p^n$
- $P(\textbf{all up}) = (1-p)^n$
- $P(\geq 1 \textbf{ down}) = 1-(1-p)^n$

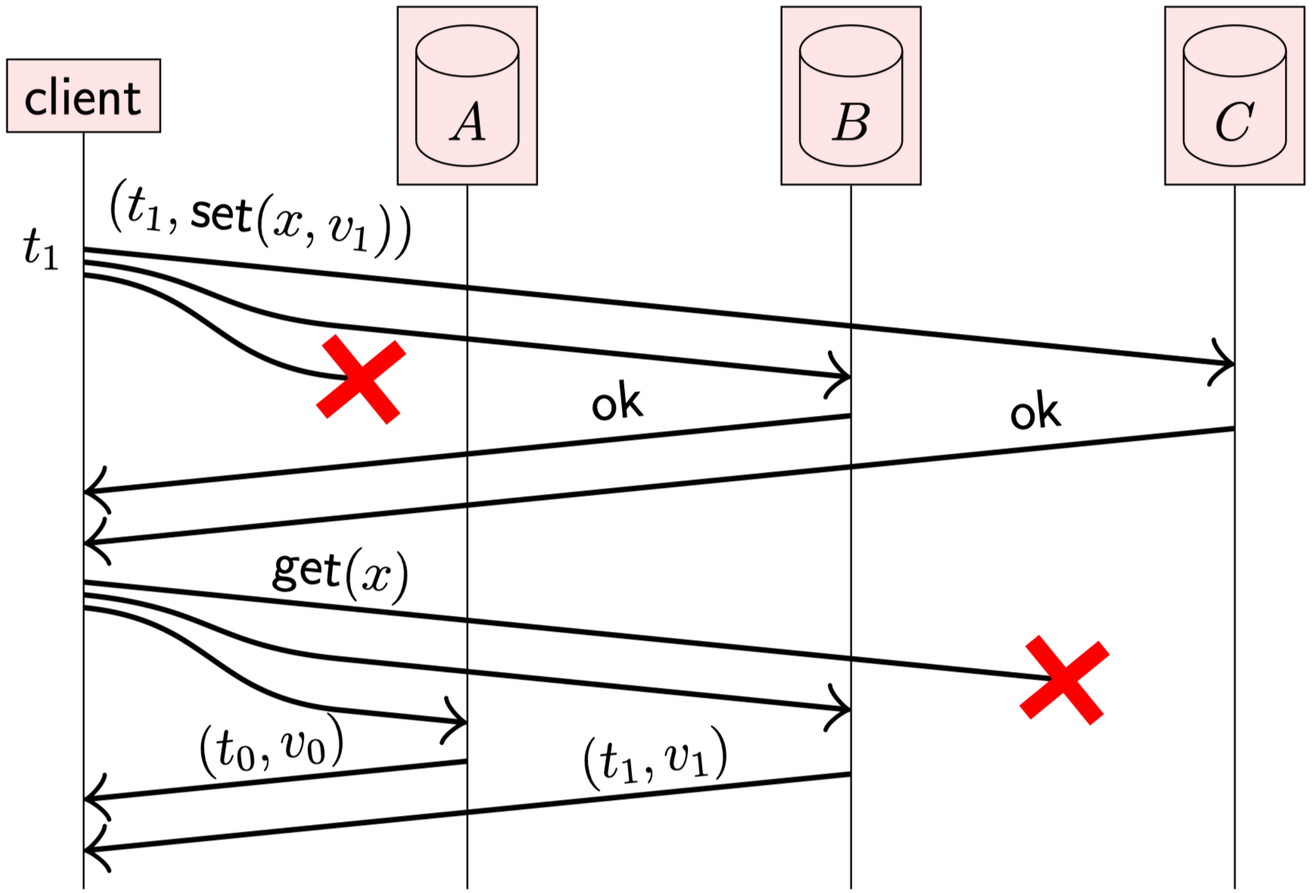
Example

Example with $p = 0.01$:

replicas n	$P(\geq 1$ faulty$)$	$P(\geq \frac{n+1}{2}$ faulty$)$	$P($all n faulty$)$
1	0.01	0.01	0.01
3	0.03	$3 \cdot 10^{-4}$	10^{-6}
5	0.049	$1 \cdot 10^{-5}$	10^{-10}
100	0.63	$6 \cdot 10^{-74}$	10^{-200}

Replication fault tolerance

- Quorum, if 2 out 3 successful



Quorum properties

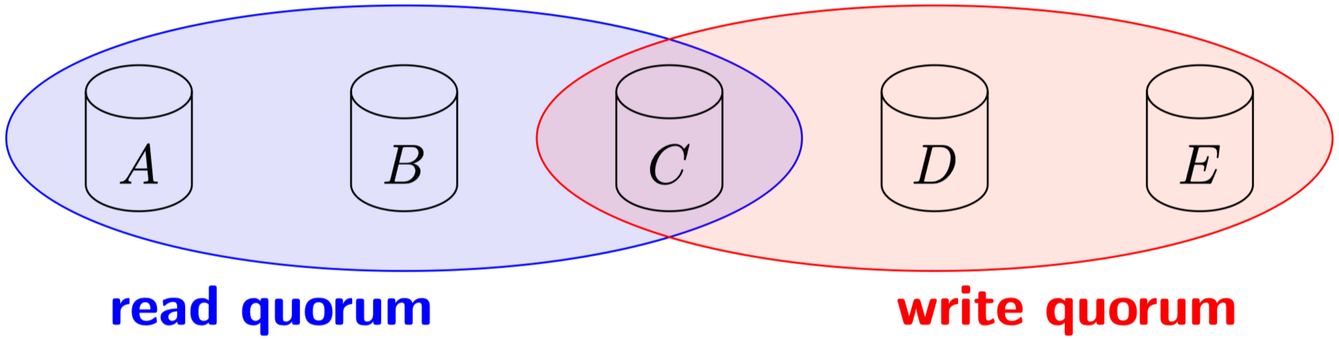
- Typical: **Majority quorum**

  at most $\lfloor \frac{n-1}{2} \rfloor$ failed nodes among n nodes

- Used widely: replication, consensus, distributed databases, shared memory, abstract data types
- Can consider: heterogeneity, workload skew, load, ...
- Quorum is used for both for **reads** and **writes**

Read and write quorums

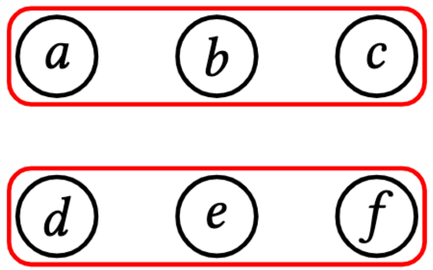
- **writes** acknowledged by w replicas
- **reads** acknowledged by r replicas
- ensure that $w + r > n$
- **result: every read will see at least one latest write**



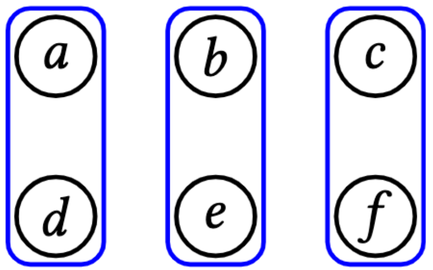
A better "Read-write" quorum

Given a set $X = \{x_1, \ldots, x_n\}$, a **read-write quorum system** [23] over X is a pair $Q = (R, W)$ where

1. R is a set of subsets of X called **read quorums**,
2. W is a set of subsets of X called **write quorums**, and
3. every read quorum intersects every write quorum. That is, for every $r \in R$ and $w \in W$, $r \cap w \neq \emptyset$.[1]

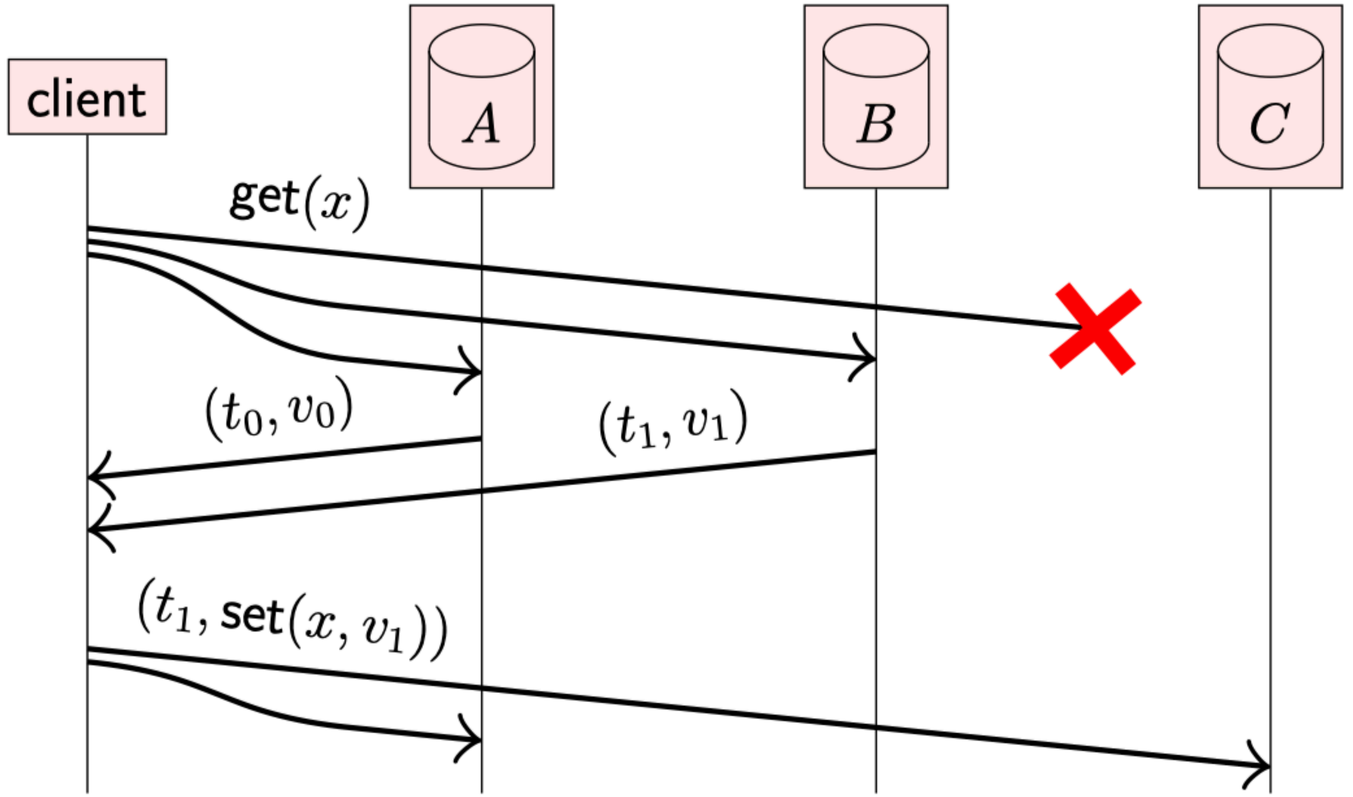


(a) Read quorums $abc + def$



(b) Write quorums $ad + be + cf$

Read repairs



Update (t_1, v_1) is more recent than (t_0, v_0) since $t_0 < t_1$.
Client helps **propagate** (t_1, v_1) to other replicas.

Using broadcast

- So far, we have used best-effort broadcast
  - it is ok to fail on some links, and re-ordering is possible
- What if we use FIFO total-order broadcast
  - Messages will be delivered **in-order** based on **happened-before** relation between events

State Machine Replication (SMR)

- (1) Using FIFO total order broadcast to broadcast every update to all replicas
  - Replicas will get the update messages in order
- (2) Updates are applied in a deterministic order on all replicas
  - e.g. according to some predetermined rules
- (1) + (2) = SMR

State Machine Replication (SMR)

- Close ideas:
  - Distributed ledgers (cryptocurrencies)
  - Smart contracts
  - Serializable transactions
- Limitations:
  - Has some update latency waiting for broadcast delivery
  - Requires **fault-tolerant total order broadcast**

Replication using weaker broadcasts

- SMR uses FIFO total order broadcast
- We can use weaker broadcasts if the updates are **commutative**
  - e.g. two updates f and g are commutative if

$$f(g(x)) = g(f(x))$$

broadcast	assumptions about state update function
total order	deterministic (SMR)
causal	deterministic, concurrent updates commute
reliable	deterministic, all updates commute
best-effort	deterministic, commutative, idempotent, tolerates message loss

Self-assesment

Why do we need replication?

What is a replica and what are the challenges in replica synchronization?

What are the differences between retry semantics?

What do we use logical clocks in replication?

What is a data tombstone?

Difference between LWW and MVR?

Contrast probabilities of all nodes being down or all nodes being up.

What is a quorum?

Why is state-machine replication useful?